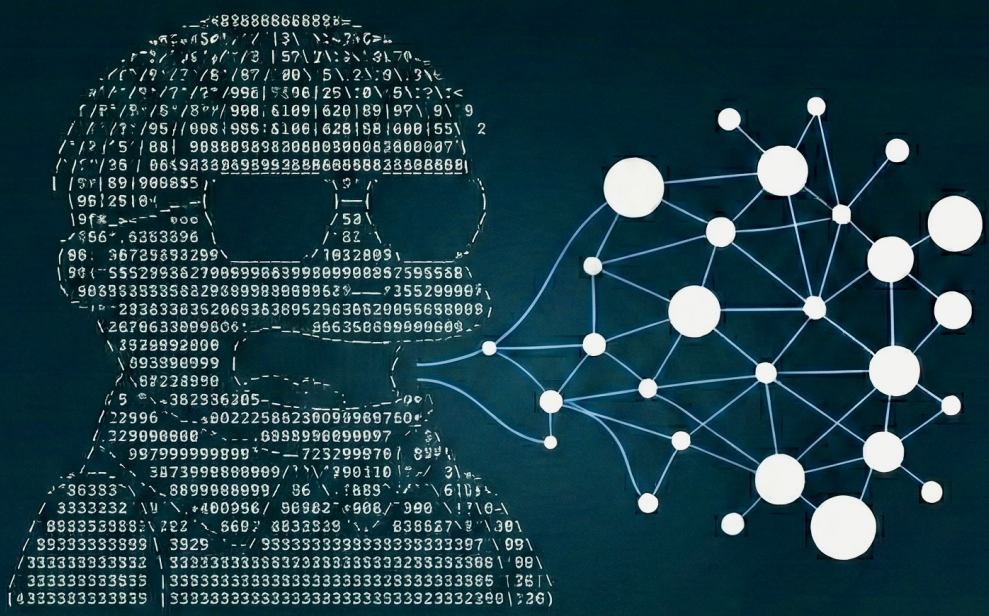
NON-DETERMINISTIC AUTONOMOUS CODING AGENTS

Building Self-Improving Systems That Ship While You Sleep



ENRICO PAPALINI

Independently published

SAMPLE

Non-Deterministic Autonomous Coding Agents

Building Self-Improving Systems That Ship While You Sleep

Copyright © 2026 Enrico Papalini

All rights reserved. No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

This is the third volume in the Non-Deterministic series, following *Non-Deterministic Software Engineering* and *Non-Deterministic Spec-Driven Development*.

ISBN: SAMPLE

Published by Independently published

Disclaimer & Security Warning

The information in this book is provided "as is" without warranty. The author and publisher shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained in this book.

Autonomous AI agents with elevated permissions can cause significant damage. The techniques in this book involve bypassing safety prompts and running agents with broad system access. Always use sandboxed environments, set strict iteration limits, and maintain the human review boundary. The author and publisher accept no responsibility for damages resulting from the implementation of techniques described herein.

Trademark Notice

Tool names, product names, and company names mentioned in this book are trademarks or registered trademarks of their respective owners.

The Simpsons, Ralph Wiggum, and all related characters and elements are trademarks of and © 20th Television (a subsidiary of The Walt Disney Company). The "Ralph Wiggum technique" and "Ralph Loop" terminology used throughout this book refer to a software development methodology created by Geoffrey Huntley that was named after the fictional character. This book is not affiliated with, endorsed by, or sponsored by 20th Television, The Walt Disney Company, or the creators of *The Simpsons*. All references to the character are for purposes of identifying the community-adopted name of the technique and constitute nominative fair use.

The views, opinions, and interpretations expressed in this book are solely those of the author and do not necessarily reflect the official positions, policies, or views of:

- Euronext or any of its subsidiaries, affiliates, or related entities
- The experts quoted or referenced herein, or their respective employers
- 20th Television, The Walt Disney Company, or the creators of *The Simpsons*

Expert quotations and insights presented in this book are the author's synthesis of publicly available materials and may not fully represent the complete context or current views of the individuals quoted. The experts have not reviewed, endorsed, or approved this book or its contents.

First Edition: February 2026

To the builders of the future,
who have the courage to let go of control,
and the vision to see what comes next.

Contents

Contents vii

Preface ix

How to Read This Book xii

The Experts Who Shaped This Book xiv

4 The Ralph Loop Pattern 3
- 4.1 Anatomy of a Ralph Loop . . . 3
  - 4.1.1 The Prompt File . . . 4
  - 4.1.2 The Plan File . . . 4
  - 4.1.3 The Completion Promise . . . 5
  - 4.1.4 Safety Limits . . . 5
  - 4.1.5 The One-Item-Per-Loop Principle . . . 5
- 4.2 The Iteration Cycle . . . 6
  - 4.2.1 Step 1: Spawn Fresh Context . . . 6
  - 4.2.2 Step 2: Orient and Select . . . 7
  - 4.2.3 Step 3: Search Before Implementing . . . 7
  - 4.2.4 Step 4: Implement . . . 7
  - 4.2.5 Step 5: Validate . . . 7
  - 4.2.6 Step 6: Commit and Update Plan . . . 8
  - 4.2.7 Step 7: Record Learnings . . . 8
  - 4.2.8 Step 8: Capture Test Intent . . . 8
  - 4.2.9 Step 9: Check Completion . . . 8
  - 4.2.10 Using Subagents for Context Management . . . 10
- 4.3 Implementation Options . . . 10
  - 4.3.1 The Bash Script Approach . . . 10
  - 4.3.2 The Stop Hook Approach . . . 11
  - 4.3.3 Enhanced Implementations . . . 12
- 4.4 Your First Ralph Loop . . . 12
  - 4.4.1 Prerequisites . . . 12
  - 4.4.2 Step 1: Create the Directory Structure . . . 13
  - 4.4.3 Step 2: Create CLAUDE.md . . . 13
  - 4.4.4 Step 3: Create the Loop Script . . . 14
  - 4.4.5 Step 4: Create Your First Plan . . . 14
  - 4.4.6 Step 5: Initialize AGENTS.md . . . 15
  - 4.4.7 Step 6: Run . . . 15
  - 4.4.8 Step 7: Review Results . . . 15
- 4.5 Understanding What Just Happened . . . 16
  - 4.5.1 What Ralph Does Well . . . 16
  - 4.5.2 What Ralph Does Not Do . . . 16
  - 4.5.3 Expect Broken Codebases . . . 17
  - 4.5.4 But Maintainability? . . . 17
- 4.6 Troubleshooting Common Issues . . . 17

5 Ralph in the Editor: Behavior Contracts and the VS Code Manual Loop **19**
5.1 The Two Problems of Autonomous Development . . . 19
5.1.1 The Behavioral Quality Problem . . . 20
5.1.2 The Context Management Problem . . . 20
5.2 The Behavior Contract File . . . 20
5.2.1 Which File to Use . . . 21
5.2.2 Why File-Based Contracts Work . . . 21
5.2.3 The Six Behavioral Rules . . . 21
5.2.4 The Session Start Checklist . . . 22
5.2.5 The Self-Improvement Loop in Detail . . . 24
5.3 The VS Code Manual Loop . . . 24
5.3.1 The Fresh-Context Principle . . . 25
5.3.2 The Fix Plan as Canonical State . . . 25
5.3.3 The Reusable Prompt Pattern . . . 26
5.3.4 VS Code Configuration . . . 26
5.4 The Combined Approach . . . 27
5.4.1 The File System as External Memory . . . 27
5.4.2 The Relationship Between Components . . . 29
5.5 Your First Combined Session . . . 30
5.5.1 One-Time Setup . . . 30
5.5.2 Populate the Behavior Contract . . . 30
5.5.3 Populate tasks/fix_plan.md . . . 31
5.5.4 Populate tasks/lessons.md . . . 32
5.5.5 Populate tasks/ralph_prompt.md . . . 33
5.5.6 Create the Shell Helper . . . 33
5.5.7 Configure VS Code . . . 35
5.5.8 Run Your First Session . . . 35
5.6 Comparing the Three Approaches . . . 36
5.7 Tuning the System . . . 37
5.7.1 Quality Gate Specification . . . 37
5.7.2 Item Granularity . . . 37
5.7.3 tasks/lessons.md Maintenance . . . 38
5.8 Troubleshooting . . . 38
5.8.1 When the Agent Goes Off Track . . . 39

Preface

It starts with a five-line bash script.

```
while :; do cat PROMPT.md | claude ; done
```

Five lines. No framework, no orchestrator, no dashboard. Just a loop that feeds a prompt to an AI agent, waits for it to finish, and does it again. And again. And again—until every task is done, every test passes, and there is nothing left to build.

That script—or something very much like it—shipped six repositories overnight at a Y Combinator hackathon. It completed a $50,000 contract for $297 in API costs. It built an entire programming language from scratch, in a language that didn't exist in any training data. And it did all of this while its operators slept.

Welcome to autonomous engineering.

The Third Book

If you have read the first two books in this series, you already know that AI-assisted development requires new thinking (*Non-Deterministic Software Engineering*) and that clear specifications are what separate productive AI collaboration from expensive chaos (*Non-Deterministic Spec-Driven Development*). This third book completes the trilogy with the question those books left open:

What if the AI never stopped working?

Not in a dystopian sense—in a practical one. What if, instead of prompting an AI and waiting for a response, you set it loose on a well-defined backlog with quality gates, memory systems, and safety boundaries, then went to bed? What if you woke up to pull requests ready for review, a progress log documenting what was tried and learned, and an agent that was smarter today than it was yesterday because it updated its own instructions?

That is what this book teaches you to build.

The Insight That Changed Everything

The core insight comes from Geoffrey Huntley, an open-source developer who, in mid-2025, started doing something that made other engineers deeply uncomfortable. He put an AI agent in a loop and walked away.

He named the technique after Ralph Wiggum from The Simpsons—"kind of dumb, kind of lovable, and it never gives up." The genius of the approach is not sophistication but stubbornness. Each iteration starts fresh: the agent reads the specifications, picks the most

important task, implements it, runs the tests, commits if they pass, updates the plan, and exits. The loop starts a new instance. No accumulated confusion. No forgotten instructions. No context rot.

Deterministically bad in an undeterministic world.

That phrase—Huntley's mantra—is counterintuitive until it clicks. It means: don't hope the AI gets it right on the first try. *Expect* failure. Make failure cheap, observable, and instructive. Build gates that reject bad work rather than scripts that prescribe good work. Then let the loop run. Given enough iterations, good enough converges on good.

By late 2025, the technique had gone viral. Anthropic released an official plugin. Ryan Carson systematized overnight execution with compound learning. The Christmas 2025 holidays saw a spike in developer experimentation so intense that the Awesome Agentic Patterns catalogue—a community-maintained repository of production-tested patterns—jumped from obscurity to nearly 2,500 GitHub stars in days. In January 2026, Steve Yegge launched Gas Town, an orchestrator for 20–30 parallel agents.

What had been one developer's experiment became an industry movement.

What You Will Build

This is not a book of theory. By the time you finish Part II, you will have a working autonomous development system running against your own codebase:

- A **Ralph Loop** that spawns fresh AI agent instances, each implementing one task from your backlog
- A **task decomposition system** that breaks features into atomic, machine-verifiable units
- A **memory layer** where agents record learnings in AGENTS.md, so Tuesday's agent is smarter than Monday's
- **Quality gates** that reject bad work automatically—tests, type checks, lints, security scans—so you can trust what the loop produces

Part III takes that working system and makes it production-grade:

- **Advanced prompting** techniques tuned for autonomous contexts
- A **pattern catalogue** drawn from nearly one hundred battle-tested agentic patterns documented by the community
- **Overnight scheduling**—the "ship while you sleep" promise, made concrete with launchd, cron, and cloud execution
- **Multi-agent orchestration** for teams ready to scale beyond a single loop
- **Safety, security, and cost management**—the discipline that separates responsible engineering from expensive disaster

Who This Book Is For

Senior developers and tech leads who have adopted AI tools and want to multiply their impact. You are comfortable with Claude Code, Cursor, or Copilot, and you want the next level.

Teams that have implemented specification-driven development from Book 2 and are ready for continuous, autonomous execution of those specifications.

Engineering managers designing autonomous development infrastructure, evaluating costs and risks, and establishing safety policies for their organizations.

You should be comfortable with AI-assisted coding, basic shell scripting, Git workflows, and automated testing. If terms like "context engineering," "AGENTS.md," and "the 70% problem" are not familiar, start with the earlier books—the concepts build on each other.

A Word of Caution

This book teaches you to run AI agents with elevated permissions, unsupervised, for hours at a time. The techniques are powerful. They are also dangerous.

The `-dangerously-skip-permissions` flag is not named that way for dramatic effect. It means the agent can read your credentials, modify your filesystem, execute arbitrary commands, and push code to your repositories. Chapter 12 is dedicated entirely to safety, security, and risk management. It is not optional reading.

Start in a sandbox. Set strict iteration limits. Never run on `main`. Review every pull request. Build trust incrementally.

The developers who thrive with autonomous systems are not the ones who are reckless. They are the ones who are methodical about risk.

Let's Begin

Geoffrey Huntley, reflecting on what happens when you first watch Ralph work, wrote: "Building software with Ralph requires a great deal of faith and a belief in eventual consistency. Ralph will test you."

He is right. Your first loop will produce broken code. Your second will be better. By your tenth, you will start to feel it—the compound effect, the growing confidence, the realization that the agent is getting smarter because you are getting better at steering it.

By your hundredth, you will wonder how you ever worked any other way.

Ready? Let's build systems that ship while you sleep.

How to Read This Book

This book is organized into four parts that take you from understanding autonomous development to running it in production.

The Recommended Path

Part I: The Autonomous Development Paradigm (Chapters 1–3)
Read sequentially. Chapter 1 establishes *why* autonomous development matters. Chapter 2 provides the philosophy that makes it work. Chapter 3 helps you assess whether you're ready. Don't skip to implementation without this foundation.
Time investment: 2–3 hours. Essential for everyone.

Part II: Building Your First Autonomous System (Chapters 4–7)
The core implementation path. Work through all four chapters sequentially, implementing as you go. Chapter 4 gets your loop running. Chapter 5 teaches you to feed it well-structured tasks. Chapter 6 gives it memory. Chapter 7 keeps it honest with quality gates. Each chapter produces artifacts you use in the next.
Time investment: 6–8 hours. Essential for everyone who wants a working system.

Part III: Scaling and Hardening (Chapters 8–12)
Takes your working system to production grade. These chapters can be read more selectively based on your needs:

- **Chapter 8** (Advanced Prompting) optimizes the prompts that drive your loops
- **Chapter 9** (Agentic Patterns) provides a reference catalogue—skim for orientation, revisit as needed
- **Chapter 10** (Scheduling) enables overnight execution
- **Chapter 11** (Multi-Agent) is advanced material for Stage 7+ readers
- **Chapter 12** (Safety) is **essential for everyone**—do not skip it

Time investment: 4–6 hours for core chapters; additional time for advanced material.

Appendices
Reference material. Use Appendix A when setting up your first system (pair with Part II). Consult Appendix B when choosing tools. Keep Appendix C handy for debugging. If you use GitHub Copilot rather than Claude Code, Appendix D adapts the book's techniques to the Copilot ecosystem—including VS Code extensions, the Copilot CLI, and subagent orchestration.
Time investment: As needed.

Reading Paths by Role

Individual Developer: Start with Part I, then work through Part II hands-on. Use Chapter 10 to set up overnight execution. Return to Chapters 8–9 to optimize. If Copilot is your primary tool, read Appendix D alongside Part II to translate the Claude Code examples.

Tech Lead: Add Chapter 11 for multi-agent understanding and Chapter 12 for team safety guidelines. Consider how autonomous loops fit into your review workflow and how the agentic patterns in Chapter 9 map to your team's challenges.

Engineering Manager: Read Chapter 3 (Developer Evolution) carefully to assess team readiness. Chapter 12 (Safety) is critical for establishing organizational policies. The cost and security implications need management attention before any team adoption.

Architect: Focus on Chapter 6 (Memory) and Chapter 11 (Multi-Agent) for system design implications. Chapter 9 (Agentic Patterns) provides the vocabulary for discussing agent architectures. Understand how autonomous development integrates with existing CI/CD infrastructure.

Quick Reference

If you need...	Go to...
Why autonomous development matters	Chapter 1 (The Revolution)
The core philosophy and principles	Chapter 2 (The Philosophy)
To assess your readiness	Chapter 3 (Developer Evolution)
To implement your first loop	Chapter 4 (The Ralph Loop Pattern)
To decompose work into tasks	Chapter 5 (Task Decomposition)
To build memory systems	Chapter 6 (Memory and Context)
To set up quality gates	Chapter 7 (Quality Gates)
Advanced prompting techniques	Chapter 8 (Advanced Prompting)
Community pattern catalogue	Chapter 9 (Agentic Patterns)
Overnight scheduling	Chapter 10 (Scheduling)
Multi-agent orchestration	Chapter 11 (Multi-Agent)
Safety and security guidance	Chapter 12 (Safety)
Complete setup walkthrough	Appendix A (Implementation Guide)
Tool comparison	Appendix B (Tool Reference)
Debugging help	Appendix C (Troubleshooting)
Ralph Loops with GitHub Copilot	Appendix D (VS Code & Copilot)

A Note on Tools and Examples

The techniques in this book are **tool-agnostic**: the Ralph Loop pattern, task decomposition, memory layers, quality gates, and scheduling strategies work with any AI coding agent that

accepts a prompt and executes commands. The underlying principles do not change when you swap one CLI for another.

That said, every practical book needs a **reference implementation**. Ours is **Claude Code**. All shell scripts, permission flags, prompt templates, and listing examples throughout the chapters use Claude Code syntax. We chose it because it was the first tool designed explicitly for autonomous terminal-based loops, it has the widest community adoption for Ralph-style workflows, and its CLI interface maps most directly to the original five-line bash script that started the movement.

If **GitHub Copilot** is your primary tool, you are not left out. **Appendix D** is a dedicated companion that translates every technique in this book to the Copilot ecosystem—VS Code extensions, the Copilot CLI, subagent orchestration, and permission profiles. Read it alongside Part II to follow the same journey with Copilot's tooling. When you encounter a Claude Code command in the main text, Appendix D provides the Copilot equivalent (for example, `claude -p "..." –dangerously-skip-permissions` becomes `copilot -p "..." –yolo`).

Other tools—Amp, Cursor, Windsurf, Codex CLI—also support autonomous loops. Appendix B compares their strengths. The patterns transfer; only the invocation syntax differs.

Code examples use realistic scenarios—authentication systems, notification features, API migrations. The specific technologies (TypeScript, React, PostgreSQL) are illustrative; the patterns apply regardless of your stack.

Companion Resources, templates, and reference materials for this book are available on GitHub at https://github.com/popoloni/non-deterministic-aca.

Scan the QR code below to access the repository directly from your mobile device:

The Experts Who Shaped This Book

The insights in these pages come from practitioners who are actively building the tools and methodologies for autonomous development—engineers shipping production systems while simultaneously defining how autonomous AI development works.

Geoffrey Huntley

Geoffrey Huntley is the creator of the Ralph Wiggum technique, the autonomous AI development methodology that went viral in late 2025. An open-source developer and engineer, Huntley developed the approach while building CURSED, a complete esoteric programming language created entirely through autonomous loops—demonstrating that AI can successfully program in a language it invented that doesn't exist in any training data.

His critical insight—that "the technique is deterministically bad in an undeterministic world"—forms the philosophical backbone of Chapter 2 and informs the entire methodology in this book. The name "Ralph" comes from The Simpsons character: "kind of dumb, kind of lovable, and it never gives up." Huntley has documented his techniques extensively at ghuntley.com and through podcast appearances, sharing both the successes (Y Combinator teams shipping $50K worth of development for $297) and the principles that make them work. His concept of "Loom"—infrastructure for evolutionary software—points toward the frontier explored in Chapter 11.

Steve Yegge

Steve Yegge is a legendary software engineer and blogger known for his influential writings on programming languages, developer productivity, and software culture. His career spans over 30 years: seven years at Amazon as Senior Manager of Software Development, thirteen years at Google as Senior Staff Software Engineer, and time at Grab before joining Sourcegraph as Head of Engineering, where he led the Cody AI coding assistant.

His contribution to autonomous development came in January 2026 with the launch of Gas Town, a multi-agent orchestrator designed to coordinate 20–30 parallel AI coding agents. His nine-stage Developer Evolution Model, which maps the progression from zero AI usage through building your own orchestrator, provides the framework for Chapter 3. Yegge co-authored *Vibe Coding* with Gene Kim and has been a vocal advocate for understanding AI coding tools as a fundamental shift in how software gets built.

Ryan Carson

Ryan Carson is a five-time founder who built, scaled, and sold three developer-focused startups over 25 years. Most notably, he founded Treehouse, an online computer science school that raised $23M in investment, scaled to over 100

employees, and educated over one million students before being acquired in 2021. He was named EY Entrepreneur of the Year 2015.
Carson developed the Compound Product methodology, which integrates Huntley's Ralph pattern with systematic learning extraction. His key contribution is the compound learning loop: agents update AGENTS.md with discovered patterns, meaning "each improvement literally makes future improvements easier." The nightly two-job sequence—compound review at 10:30 PM followed by auto-compound implementation at 11:00 PM—which is covered in Chapter 10, comes directly from his implementation. His work on snarktank/ralph and snarktank/compound-product provides the practical foundation for much of Part II.

Addy Osmani

Addy Osmani is a Director at Google Cloud AI with nearly 14 years of experience leading developer experience on the Chrome team, where he built DevTools, Lighthouse, and Core Web Vitals. Author of numerous influential books including *Beyond Vibe Coding*, *Learning JavaScript Design Patterns*, and *Leading Effective Engineering Teams*, he has shaped how millions of developers build web applications. His observation that "many people will throw the kitchen sink at the LLM and ask it to build all the requirements at once. That approach does not necessarily work well" informs Chapter 5's task decomposition methodology. His insight about the "sweet spot" of 2–3 parallel agents, beyond which diminishing returns set in, provides grounding for Chapter 11's multi-agent discussion. Throughout the book, Osmani's practical wisdom about what actually works at scale keeps the methodology grounded in real-world constraints.

Together, these experts provide the complete picture: Huntley created the core technique and philosophy; Yegge mapped the developer evolution and built multi-agent orchestration; Carson systematized compound learning and overnight execution; Osmani provided the practical wisdom about scaling and constraints. This book synthesizes their insights into a comprehensive guide for building autonomous development systems.

Part I — Chapters Not Included in This Sample

Chapter 1: The Continuous Development Revolution *Not included in this sample.*

Chapter 2: The Philosophy of Deterministic Failure *Not included in this sample.*

Chapter 3: The Developer Evolution Model *Not included in this sample.*

4 The Ralph Loop Pattern

Ralph is a Bash loop. A simple while true that repeatedly feeds an AI agent a prompt file, allowing it to iteratively improve its work until completion.

— Geoffrey Huntley, Creator of the Ralph Wiggum Technique

Chapter 3 helped you assess your readiness for autonomous development. Now we build. This chapter teaches the fundamental loop pattern that underlies all autonomous development—from the simplest bash script to sophisticated orchestration systems.

The name comes from Ralph Wiggum of The Simpsons—perpetually confused, always making mistakes, but never stopping. As Geoffrey Huntley describes it: "It's kind of dumb, it's kind of lovable, and it never gives up." This childlike persistence is exactly what inspired the technique's name. With a Ralph Loop, the AI is no longer allowed to simply exit. It is forced to keep working until the task is truly finished.

At its core, a Ralph Loop is deceptively simple. Huntley describes it as nothing more than a bash loop—a straightforward `while true` that repeatedly feeds an AI agent a prompt file. Yet this simplicity masks profound implications for how we build software. The technique went viral in late 2025, producing the dramatic results documented in Chapter 1's case study.

This chapter teaches you to build and run your own Ralph Loops. By the end, you will understand not just the mechanics but the principles that make this pattern so effective.

4.1 Anatomy of a Ralph Loop

The minimal Ralph Loop can be expressed in a single line of bash:

```
while :; do cat PROMPT.md | claude ; done
```

This line captures the essential pattern: take a prompt file, feed it to an AI agent, wait for completion, then do it again. The colon after `while` is bash shorthand for "true"—the loop runs forever until manually stopped or until a completion condition is met.

Warning

The one-liner above is illustrative only—do not use it in production. It lacks safety limits, error handling, and completion detection. See Section 4.4 for a practical implementation and Appendix A for the complete production-ready script.

A practical Ralph Loop requires several components working together: a prompt file containing instructions, a plan file tracking tasks, a completion promise signaling when to stop, and safety limits preventing runaway execution.

4.1.1 The Prompt File

The prompt file (typically `PROMPT.md` for Amp or `CLAUDE.md` for Claude Code) contains everything the AI needs to understand its task and context. Unlike interactive prompting where you build context through conversation, the prompt file must be self-contained—each iteration starts fresh with no memory of previous runs.

A well-structured prompt file includes orientation instructions telling the agent where to find project context, task selection logic directing the agent to pick the next incomplete task, implementation guidance covering coding standards and patterns, quality requirements specifying which checks must pass before committing, and completion signals defining what to output when all work is done.

Geoffrey Huntley on Prompt Evolution

"There is no such thing as a perfect prompt... The prompt you start with won't be the prompt you end with—it has evolved through continual tuning based on observation of LLM behaviour."

4.1.2 The Plan File

The plan file serves as the agent's work queue. Each iteration, the agent reads this file, selects the highest-priority incomplete item, implements it, and updates the plan to reflect completion. Two formats are commonly used: a markdown-based `fix_plan.md` with checkbox syntax (`- [ ]` / `- [x]`) for simplicity, and a structured `prd.json` with explicit pass/fail states for programmatic tracking. Chapter 6 provides complete schemas, examples, and guidance for both formats.

Key Point

The plan is disposable. Huntley notes: "I have deleted the TODO list multiple times... if I throw the TODO list out, you might be asking, 'Well, how does it

> know what the next step is?' Well, it's simple. You run a Ralph loop with explicit instructions to generate a new TODO list." When Ralph goes off track, regenerate the plan rather than trying to salvage a corrupted one.

4.1.3 The Completion Promise

How does the loop know when to stop? Through a **completion promise**—a specific string the agent outputs when all work is finished. The standard convention uses XML-style tags:

```
<promise>COMPLETE</promise>
```

The loop script watches for this string in the agent's output. When detected, the loop exits gracefully. Without a completion promise, you would need to manually stop the loop or rely solely on iteration limits.

> **⚠ Warning**
>
> The completion promise uses exact string matching. You cannot use it for multiple completion conditions like "SUCCESS" versus "BLOCKED." Always rely on `-max-iterations` as your primary safety mechanism, with the completion promise as the normal exit path.

4.1.4 Safety Limits

Production Ralph Loops include multiple safety mechanisms:

- **Maximum iterations** prevents infinite loops when tasks prove impossible or the agent gets stuck. A typical starting point is 10–25 iterations for focused features.
- **Iteration counter** tracks progress and enables the script to exit when limits are reached.
- **Timeout handling** catches cases where the agent hangs without producing output.
- **Cost monitoring**, while not always built into the loop itself, should be tracked externally to prevent runaway API charges.

4.1.5 The One-Item-Per-Loop Principle

This principle is so important that Huntley repeats it emphatically: "One item per loop. I need to repeat myself here—one item per loop."

The reasoning connects directly to context window management. You have approximately 176k usable tokens of context window to work with (after model and tool overhead), and the more you use, the worse the outcomes. By focusing on a single item, you keep context lean and the agent focused.

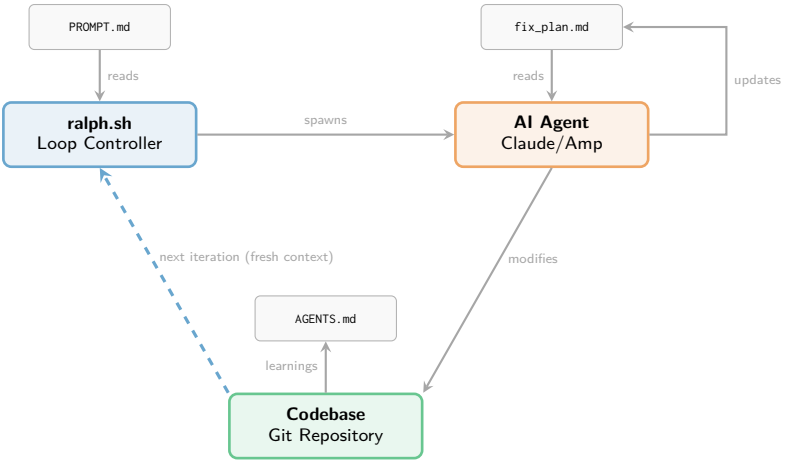


Figure 4.1: Anatomy of a Ralph Loop showing the relationship between the loop controller script, AI agent, and persistent artifacts. Each iteration spawns a fresh agent instance that reads the plan, modifies the codebase, updates the plan status, and records learnings.

You may relax this restriction as the project progresses, but if outcomes start degrading, narrow back down to one item. This constraint is the "Achilles' heel" that you tune—too many items causes confusion; too few slows progress unnecessarily.

A surprising aspect of the Ralph technique is trusting the agent to decide what is most important:

> "*This might seem wild, but you also need to trust Ralph to decide what's the most important thing to implement. LLMs are surprisingly good at reasoning about what is important to implement and what the next steps are.*"

This requires faith in eventual consistency—knowing that even if Ralph makes suboptimal choices, the loop will eventually work through all items.

4.2 The Iteration Cycle

Each iteration of a Ralph Loop follows a precise sequence. Understanding this sequence is essential for debugging issues and optimizing performance.

4.2.1 Step 1: Spawn Fresh Context

The loop spawns a completely fresh AI agent instance. This is not resuming a conversation—it is starting over with a blank slate. The agent has no memory of previous iterations except what is explicitly provided through files.

This fresh-context approach is fundamental to Ralph's reliability. As discussed in Chapter 2, starting fresh each time prevents the "context rot" that makes long-running sessions unpredictable.

4.2.2 Step 2: Orient and Select

The agent reads its prompt file and orients itself to the project. Huntley's prompt structure emphasizes deterministic context allocation—loading the same files in the same order every loop:

```
0a. Study specs/* to learn about the specifications
0b. The source code is in src/
0c. Study fix_plan.md
```

With context established, the agent selects the next task—the highest-priority incomplete item from the plan.

4.2.3 Step 3: Search Before Implementing

A critical step that prevents duplicate work: the agent must search the codebase before assuming something is not implemented. Huntley identifies this as a common failure mode:

> "*A common failure scenario for Ralph is when the LLM runs ripgrep and comes to the incorrect conclusion that the code has not been implemented.*"

The fix is explicit instruction in your prompt:

```
Before making changes, search codebase
(don't assume an item is not implemented) using subagents.
Think hard.
```

4.2.4 Step 4: Implement

The agent implements the selected task. During implementation, the agent should follow patterns established in `AGENTS.md`, make incremental changes rather than massive rewrites, and consider edge cases specified in acceptance criteria.

Claude has an inherent bias toward minimal and placeholder implementations. Huntley addresses this directly in his prompts:

```
DO NOT IMPLEMENT PLACEHOLDER OR SIMPLE IMPLEMENTATIONS.
WE WANT FULL IMPLEMENTATIONS.
```

4.2.5 Step 5: Validate

Before committing, the agent runs quality checks. This is where backpressure—the core concept from Chapter 2—does its work.

```
After implementing functionality or resolving problems,
run the tests for that unit of code that was improved.
```

The key insight is that *anything* can be wired in as backpressure to reject invalid code generation: test suites, type checkers, security scanners, static analyzers, linters. If ANY check fails, the agent should attempt to fix the issue and re-run validation.

4.2.6 Step 6: Commit and Update Plan

With validation passing, the agent commits the change to git:

```
When the tests pass, update the fix_plan.md,
then add changed code and fix_plan.md with "git add -A"
then do a "git commit" with a message that describes the changes.
After the commit do a "git push".
```

The plan update is crucial—marking items complete ensures the next iteration will not re-select this task.

4.2.7 Step 7: Record Learnings

This step distinguishes Ralph from simpler loop patterns. The agent should update `AGENTS.md` with discovered patterns:

```
When you learn something new about how to run the compiler
or examples, make sure you update AGENTS.md using a subagent
but keep it brief.
```

This creates compound learning—future iterations benefit from past discoveries.

4.2.8 Step 8: Capture Test Intent

When writing tests, Huntley emphasizes capturing the *meaning* and *importance* of each test. Because each loop has a fresh context window, the reasoning behind a test is lost unless explicitly captured. These notes help the LLM decide if a test is still relevant in future iterations.

4.2.9 Step 9: Check Completion

Finally, the agent checks whether all tasks are complete. If every item in the plan is done, the agent outputs the completion promise. If tasks remain, the agent simply exits normally, and the loop spawns a fresh instance to continue.

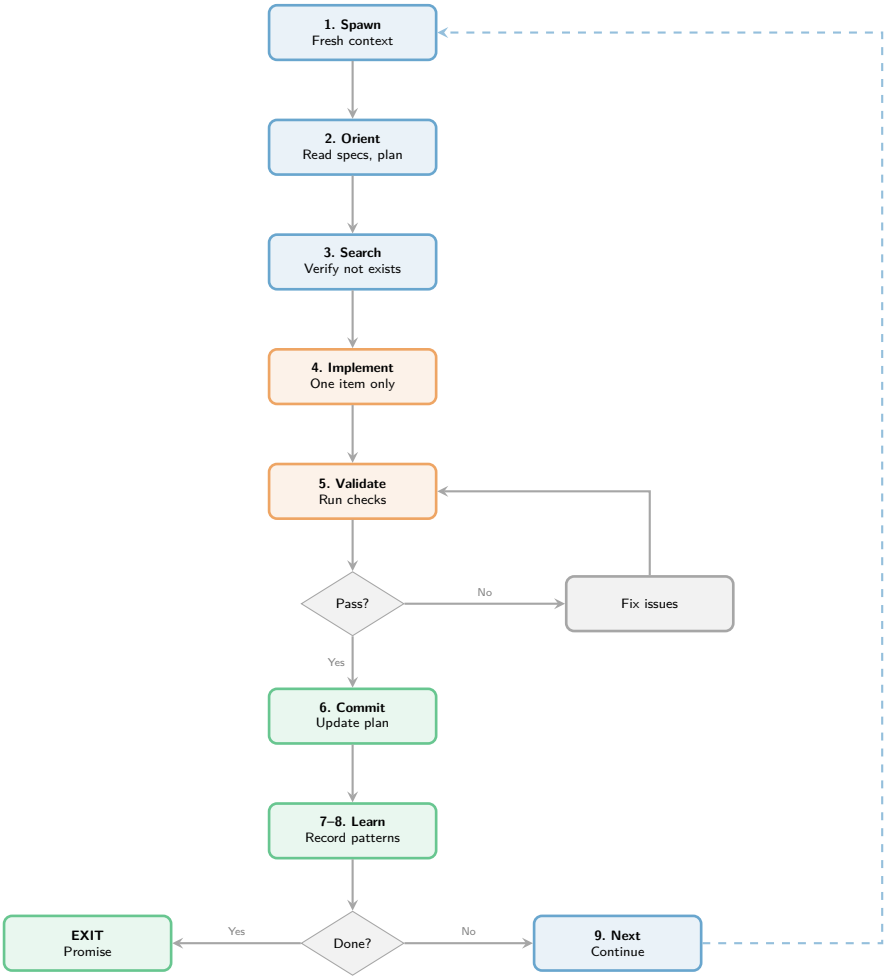


Figure 4.2: The complete iteration cycle of a Ralph Loop. Each iteration spawns fresh, searches before implementing, validates via backpressure, commits on success, records learnings, and either exits or loops for the next task.

4.2.10 Using Subagents for Context Management

A key technique in Huntley's approach is using **subagents** to extend the effective context window without polluting the primary context:

> "*Ralph requires a mindset of not allocating to the primary context window. Instead, spawn subagents. Your primary context window should operate as a scheduler, scheduling other subagents to perform expensive allocation-type work.*"

The prompt controls parallelism explicitly:

```
You may use up to 500 parallel subagents for all operations
but only 1 subagent for build/tests.
```

This prevents bad backpressure from parallel validation—if many subagents run the build simultaneously, the results flood back and overwhelm the primary context.

4.3 Implementation Options

Several implementations of the Ralph Loop pattern exist, each with different trade-offs.

4.3.1 The Bash Script Approach

The original Ralph implementation is a bash script that externally loops, spawning fresh AI agent processes. The snarktank/ralph repository provides a battle-tested version.

The bash approach offers several advantages: it works with any AI tool that has a CLI, requires no special plugins, is easy to understand and modify, and runs on any Unix-like system. Most importantly, it enforces fresh context each iteration—the key reliability mechanism.

A simplified script for illustration (see Appendix A for the complete production version):

> **Warning**
>
> The `-dangerously-skip-permissions` flag bypasses Claude's permission system entirely, allowing the agent to execute arbitrary commands, read any file, and make network requests without approval. This is necessary for autonomous operation but requires a sandbox to contain the blast radius. See Chapter 13 for comprehensive sandbox strategies and security requirements before running this in any environment.

Note: The flag name varies by tool. Claude Code uses `-dangerously-skip-permissions`; Amp uses `-dangerously-allow-all`; Codex uses `-full-auto`. The semantics are identical: unattended execution without permission prompts. Regardless of the flag name, always sandbox the process.

```bash
#!/bin/bash
SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
MAX_ITERATIONS=${1:-25}

for ((i=1; i<=MAX_ITERATIONS; i++)); do
    echo "=== Iteration $i of $MAX_ITERATIONS ==="

    OUTPUT=$(claude --dangerously-skip-permissions \
        --print < "$SCRIPT_DIR/CLAUDE.md" 2>&1 | tee /dev/stderr)

    if echo "$OUTPUT" | grep -q "<promise>COMPLETE</promise>"; then
        echo "Ralph completed all tasks!"
        exit 0
    fi

    sleep 2
done
echo "Reached maximum iterations"
exit 1
```

4.3.2 The Stop Hook Approach

Claude Code's official ralph-wiggum plugin takes a different approach. Instead of an external loop, it installs a **stop hook** inside your Claude session:

```
/ralph-loop "Implement the authentication system" \
    --completion-promise "DONE" \
    --max-iterations 20
```

The plugin intercepts Claude's attempts to exit via a lifecycle hook. When Claude tries to stop, the hook checks whether the completion promise has been output. If not, it returns exit code 2 along with the original prompt, causing Claude to continue working rather than exiting. If the completion promise is detected, exit code 0 allows normal termination. If the iteration limit is reached, exit code 1 forces a stop.

Geoffrey Huntley on the Official Plugin

"The official Anthropic plugin misses a key point of Ralph which is not 'run forever' but 'carve off small bits of work into independent context windows.' The bash loop approach forces fresh context each iteration, which is fundamental to reliability."

The stop hook approach is convenient for quick experiments but has a fundamental limitation: the session continues rather than restarting, so context accumulates rather than resetting—precisely the problem that fresh-context approaches solve (see Chapter 2). For production use, most practitioners prefer external loops that spawn truly fresh instances.

4.3.3 Enhanced Implementations

Several community implementations add production-ready features:

frankbria/ralph-claude-code adds intelligent exit detection requiring both completion indicators and an explicit exit signal, rate limiting with hourly reset to manage API costs, and a circuit breaker with advanced error detection to prevent runaway loops.

ralph-orchestrator (mikeyobrien) provides a Rust-based implementation with hat-based orchestration for specialized agent personas, Telegram integration for human-in-the-loop interaction, and a terminal UI for monitoring running loops.

Table 4.1: Comparing Ralph Loop Implementations

Feature	snarktank/ralph	Anthropic Plugin	ralph-orchestrator
Fresh context	Yes	No	Yes
Dependencies	Minimal (bash, jq)	Claude Code	Rust runtime
Learning curve	Low	Low	Medium
Production features	Basic	Basic	Advanced
Best for	Getting started	Quick experiments	Production use

4.4 Your First Ralph Loop

Let us build a working Ralph Loop from scratch. This walkthrough uses the bash script approach with Claude Code, but the principles apply to any implementation.

4.4.1 Prerequisites

Before building your first Ralph Loop, ensure your environment is ready.

Required tools:

```
# Claude Code CLI
npm install -g @anthropic-ai/claude-code
claude --version  # Expected: 2.0.76 or later

# jq for JSON manipulation
brew install jq   # macOS
# or: sudo apt-get install jq  # Linux

# Git configured
git config user.name   # Must return a value
git config user.email  # Must return a value
```

Project requirements:

Ralph works best with projects that have existing quality infrastructure. Your project needs a runnable test suite, linting configured, type checking (for typed languages), and a clean git state on a feature branch.

Key Point

For dynamically typed languages, Huntley stresses the critical importance of wiring in a static analyzer or type checker. For Python, use `pyright` or `mypy`. For JavaScript, use TypeScript or ESLint with type-aware rules. Without this backpressure, you will "run into a bonfire of outcomes."

4.4.2 Step 1: Create the Directory Structure

```
mkdir -p scripts/ralph
mkdir -p specs
```

4.4.3 Step 2: Create CLAUDE.md

Create `scripts/ralph/CLAUDE.md` with your prompt. This template requires customization for your project's quality check commands:

```
# Ralph Loop Instructions

You are working in an autonomous loop. Each iteration:
1. Read the current state
2. Implement ONE task (only one!)
3. Validate your work
4. Commit if successful

## Orientation
- Read AGENTS.md for project patterns
- Read specs/* for project specifications
- Read fix_plan.md for the current plan

## Task Selection
Load fix_plan.md, find highest-priority incomplete item.
Only ONE item per iteration.

## Implementation
- Search before coding---do NOT assume anything missing
- Make minimal, focused changes following existing patterns
- Write FULL implementations, not placeholders

## Validation
Run ALL quality checks (typecheck, lint, test).
If ANY check fails, fix and re-run. Do NOT commit failing code.

## Commit
```

```
If all checks pass: update fix_plan.md, git add, commit, push.

## Completion Check
If ALL items are complete, output: <promise>COMPLETE</promise>
Otherwise, exit normally---the loop continues.
```

4.4.4 Step 3: Create the Loop Script

Create scripts/ralph/ralph.sh and make it executable:

```
#!/bin/bash
set -e

SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
PROJECT_ROOT="$(cd "$SCRIPT_DIR/../.." && pwd)"
MAX_ITERATIONS=${1:-25}

cd "$PROJECT_ROOT"

echo "Starting Ralph Loop (max: $MAX_ITERATIONS iterations)"

for ((i=1; i<=MAX_ITERATIONS; i++)); do
    echo "========================================"
    echo "Iteration $i of $MAX_ITERATIONS - $(date)"
    echo "========================================"

    OUTPUT=$(claude --dangerously-skip-permissions \
        --print < "$SCRIPT_DIR/CLAUDE.md" 2>&1 | tee /dev/stderr) ||
     true

    if echo "$OUTPUT" | grep -q "<promise>COMPLETE</promise>"; then
        echo "SUCCESS! Ralph completed at iteration $i"
        exit 0
    fi

    echo "Iteration complete. Continuing..."
    sleep 2
done

echo "WARNING: Reached max iterations ($MAX_ITERATIONS)"
exit 1
```

```
chmod +x scripts/ralph/ralph.sh
```

4.4.5 Step 4: Create Your First Plan

Create fix_plan.md:

```
# Implementation Plan
```

```

## High Priority

- [ ] Add greeting utility function
  - Create src/utils/greeting.ts
  - Export function greet(name: string): string
  - Returns 'Hello, {name}!' format
  - Add unit test covering empty string edge case

- [ ] Add farewell utility function
  - Add function farewell(name: string): string
  - Returns 'Goodbye, {name}!' format
  - Add unit tests

## Completed

(Items will be moved here as completed)
```

4.4.6 Step 5: Initialize AGENTS.md

Create `AGENTS.md` with project-specific guidance:

```
# Project Patterns

## Code Style
- Use TypeScript for all new files
- Add JSDoc comments for public functions

## Testing
- Use Jest for unit tests
- Place tests adjacent to source (*.test.ts)
- Document WHY each test exists

## Build Commands
- Type check: npm run typecheck
- Lint: npm run lint
- Test: npm test
```

4.4.7 Step 6: Run

```
git checkout -b feature/ralph-demo
./scripts/ralph/ralph.sh 10
```

Watch as Ralph iterates through your tasks. You should see it search the codebase, create files, write tests, run validation, and commit successful changes.

4.4.8 Step 7: Review Results

```
cat fix_plan.md      # Check plan status
cat AGENTS.md        # Review learnings
git log --oneline -10  # See commits
git diff main...HEAD   # Examine changes
```

Warning

Your first run will likely reveal issues—perhaps quality checks have different names, or the agent misunderstands your codebase. This is normal. Review what happened, update your prompt with corrections, and try again. As Huntley says, "Ralph gets tuned—like a guitar."

4.5 Understanding What Just Happened

If your first Ralph Loop succeeded, you have witnessed something remarkable: an AI agent working autonomously through multiple tasks, validating its own work, and shipping code without your intervention.

4.5.1 What Ralph Does Well

Ralph excels at **well-specified implementation work**. When tasks have clear acceptance criteria and tests can verify correctness, Ralph churns through work efficiently. The compound effect is powerful: each iteration learns from the last, patterns discovered early inform later work.

Ralph also excels at **toil elimination**. Migrations, test coverage expansion, documentation generation, code standardization—these are tasks with clear before/after states and mechanical execution.

4.5.2 What Ralph Does Not Do

Ralph does not replace engineering judgment. It implements specifications; it does not write them. The quality of Ralph's output depends entirely on the quality of your specifications, tests, and quality gates.

As Huntley discovered building CURSED: "A big, hard lesson for me was that my specification for the lexer defined a keyword twice for two opposing scenarios, which resulted in a lot of time wasted. Ralph was doing stupid shit, and I guess it's easy to blame the tools instead of the operator."

Key Point

Ralph is a force multiplier, not a replacement. It amplifies your specifications, your tests, your patterns. Engineers who write clear specs and maintain solid test suites will see Ralph dramatically accelerate their work. Engineers who skip these fundamentals will find Ralph generates confident nonsense at impressive speed.

4.5.3 Expect Broken Codebases

Huntley is honest about this reality:

> "*Yep, it's true, you'll wake up to a broken codebase that doesn't compile from time to time. This is where you need to put your brain on. Is it easier to do a* `git reset -hard` *and kick Ralph back off again? Or do you need to come up with another series of prompts to rescue Ralph?*"

The key insight is that *any problem created by AI can be resolved through a different series of prompts*. Sometimes the fix is to regenerate the plan. Sometimes it is to add a new "sign" to the prompt. Sometimes it is to reset and start fresh.

4.5.4 But Maintainability?

A common objection is concern about code maintainability. Huntley reframes the question:

> "*When I hear that argument, I question 'by whom'? By humans? Why are humans the frame for maintainability? Aren't we in the post-AI phase where you can just run loops to resolve/adapt when needed?*"

This is a philosophical shift worth considering. If AI can generate code, it can also modify code. The traditional assumption that code must be optimised for human reading may need updating—though this remains a contested point in the community.

4.6 Troubleshooting Common Issues

A comprehensive troubleshooting guide with detailed diagnostics is provided in Appendix C.

Chapter Summary

- A Ralph Loop is fundamentally simple: a script that repeatedly spawns fresh AI agent instances to work through a plan
- The core cycle is **Spawn** → **Orient** → **Search** → **Implement** → **Validate** → **Commit** → **Learn** → **Check Completion**
- **One item per loop**—this constraint is critical for reliable outcomes
- **Search before implementing**—do not assume code does not exist

Table 4.2: Common Ralph Loop Issues and Solutions

Problem	Solution
Loop exits immediately	Check that `fix_plan.md` exists with uncompleted items
Quality checks always fail	Update prompt to match your actual tooling commands
Agent commits failing code	Add explicit verification steps; consider pre-commit hooks
Same task attempted repeatedly	Strengthen "search before implementing" instructions
Placeholder implementations	Add explicit "NO PLACEHOLDER" instructions to prompt
Agent stuck on one issue	Add guidance: "After 3 attempts, document blocker and move on"
Costs running high	Reduce `max-iterations`; ensure tasks are appropriately sized
Duplicate implementations	Code search is non-deterministic; add explicit "search before implementing" and "update existing, never recreate" instructions
Context fills with errors	Use a separate AI session to create a prioritised fix plan from the error output, then feed it back to Ralph

- Fresh context each iteration is the foundation of reliable autonomous behavior (see Chapter 2)
- Use **subagents** for expensive operations while keeping validation to a single subagent
- The plan is disposable—regenerate it when Ralph goes off track
- Multiple implementations exist: bash scripts for simplicity, stop hooks for convenience, enhanced tools for production
- Your first loop will reveal issues—tune prompts based on observation, "like tuning a guitar"
- Ralph amplifies your specifications and tests; quality of output depends on quality of inputs

---

What's Next: A Ralph Loop is only as good as the tasks it executes. The next chapter dives deep into task decomposition—how to break features into atomic, verifiable units that AI can implement reliably. You will learn to write specifications that leave no room for misinterpretation and acceptance criteria that machines can actually verify.

5 Ralph in the Editor: Behavior Contracts and the VS Code Manual Loop

Ralph does not need a terminal. He needs a plan file, a lessons file, and one rule: always start a new chat when the item is done.

— ***VS Code Ralph practitioner note***

Chapter 4 built the Ralph Loop from scratch—a bash script that drives an AI agent through your task list with the mechanical persistence of a confused but determined child. This chapter addresses a different reality: most developers live in VS Code, not the terminal. It introduces two complementary techniques—the ***behavior contract*** *and the* ***VS Code manual loop****—and then shows how combining them produces a system that achieves the same reliability guarantees as the original Ralph Loop, without requiring a CLI or any tooling beyond the editor you already have open.*

The original Ralph Loop is elegant in its Unix-ness: a bash script, a prompt file, a fresh process per iteration. But the majority of working developers spend their days in VS Code, and GitHub Copilot Agent Mode has become one of the most widely deployed AI coding assistants in the world. The question is not whether to apply the Ralph Loop's principles—it is how to apply them inside the editor you already have open.

Two distinct problems need solving, and they are independent. The first is *behavioral quality*: ensuring the agent plans before acting, verifies before declaring done, and learns from corrections rather than repeating them. The second is *context management*: ensuring that quality does not degrade as a session accumulates history. The original Ralph Loop solves context management structurally—each bash iteration spawns a fresh process—but says nothing about behavioral quality. A behavior contract file solves behavioral quality explicitly but says nothing about context management. The combined approach addresses both.

5.1 The Two Problems of Autonomous Development

Before building the solution it is worth understanding precisely what each problem looks like in practice.

5.1.1 The Behavioral Quality Problem

Left to their own devices, AI coding agents exhibit a predictable set of failure modes. They implement without planning, shipping code that solves the immediate request while introducing architectural debt. They declare tasks complete without verification, marking items done when tests still fail. They repeat the same mistakes across sessions, making the identical wrong assumption about your codebase on Monday that they made on Friday. They reach for expedient but incorrect solutions when the right solution requires understanding context they have not loaded.

These are not intelligence failures—they are behavioral failures. The agent is capable of better; it simply has no contract telling it what "better" looks like in your specific project.

5.1.2 The Context Management Problem

The second problem is more subtle. AI agents are not stateless—they reason over a context window that accumulates everything: conversation history, tool call outputs, file contents, error messages. As a session grows longer, several things happen simultaneously. The agent's effective attention spans fewer tokens of actual content. Early context gets compressed or dropped. The agent begins reasoning from stale fragments of earlier content rather than the current state of the codebase. Decision quality degrades, often imperceptibly at first and then sharply.

Huntley identified this pattern in the original Ralph Loop and solved it structurally: each bash iteration spawns a completely fresh process. There is no accumulated context because there is no accumulated session. The tradeoff is that each iteration must re-orient from scratch—which is precisely why the plan file and `AGENTS.md` exist.

The VS Code equivalent of this structural solution is starting a *new chat session* for each work item. It is less automated but mechanically identical: fresh context, no history, full re-orientation from files on disk.

Key Point

Context management and behavioral quality are independent problems requiring independent solutions. Fixing one does not fix the other. A fresh-context session with no behavioral rules will still make planning mistakes. A well-behaved agent in a degraded context will still produce worse output than the same agent starting fresh. You need both.

5.2 The Behavior Contract File

A **behavior contract** is a markdown file placed in your project that the agent reads at the start of every session. It does not define *what* to build—that lives in your task files. It defines *how* the agent should behave while building anything.

Think of it as the cultural norms of your codebase, written down for an agent that has no prior exposure to your team. A new human engineer absorbs these norms through code review, pairing, and conversation. An AI agent absorbs them through the contract you write upfront and the file it reads before touching anything.

5.2.1 Which File to Use

Different tools read different files. Before writing your contract, you need to know which file your tool loads automatically.

GitHub Copilot Agent Mode reads `.github/copilot-instructions.md` automatically for custom instructions at every session. This is the correct location for your behavior contract when working with Copilot. Claude Code reads `CLAUDE.md` from the project root. Amp reads `AGENTS.md`. Several tools, including recent versions of Copilot with agent extensions, will also scan `AGENT.md` in the project root.

> **⚠ Warning**
>
> Do not assume your tool reads `AGENT.md` automatically without confirming. If you place your contract in the wrong file, the agent will behave as if no contract exists—silently, with no error. Verify by including a distinctive instruction (for example, always greet the user with the project name) and checking that a fresh session follows it before relying on the contract for real work.

This chapter uses the filename `AGENT.md` throughout because it is tool-agnostic shorthand. For GitHub Copilot specifically, create `.github/copilot-instructions.md` with the same content. For Claude Code, create `CLAUDE.md`. The content is identical; only the path differs.

5.2.2 Why File-Based Contracts Work

The mechanism works because the contract file is loaded fresh each session—it is not part of conversation history, it is part of the fixed context that every session starts with.

Conversation history degrades over time and disappears at session boundaries. File-based instructions do not. Every new session reads the same contract from disk and starts from the same behavioral baseline, regardless of how many sessions have come before it. This persistence is the foundation on which cross-session reliability is built.

5.2.3 The Six Behavioral Rules

A complete behavior contract covers six areas of concern:

Rule 1: Plan Mode Default. The agent enters plan mode for any non-trivial task before writing code. "Non-trivial" means three or more implementation steps, or any decision with architectural implications. Plans are written to `tasks/todo.md` as checkable items

before implementation begins. If something goes wrong mid-implementation, the correct response is to stop and re-plan—not to push through.

Rule 2: Subagent Strategy. When a task requires research, exploration, or parallel analysis, the agent spawns **subagents** rather than loading everything into primary context. The primary context window acts as a scheduler; expensive operations happen in subagents. This mirrors Huntley's subagent guidance for bash Ralph, applied to Copilot's parallel processing capabilities.

Rule 3: Self-Improvement Loop. After any correction from the user, the agent updates `tasks/lessons.md` with the pattern. Each lesson captures what happened, why it happened, and the specific rule that prevents recurrence. At the start of each session, the agent reads this file before doing anything else. This creates cross-session learning without relying on conversation history.

Rule 4: Verification Before Done. A task is never marked complete without demonstrated correctness. The agent runs tests, checks logs, and asks itself whether a staff engineer would approve the change. Diffing behavior between the original and modified versions is encouraged for non-trivial changes.

Rule 5: Demand Elegance (Balanced). For non-trivial changes, the agent pauses to ask whether a more elegant solution exists. For simple fixes, this step is skipped to avoid over-engineering. The goal is preventing both hacky expedience and unnecessary complexity.

Rule 6: Autonomous Bug Fixing. When given a bug report, the agent fixes it without requesting hand-holding. It consults logs, errors, and failing tests autonomously, then resolves them without context-switching costs to the user.

5.2.4 The Session Start Checklist

The most important element of any behavior contract is the session start checklist: the set of actions the agent must take *before touching any code*:

```
## Session Start --- Do This First, Every Time

Before touching any code:
1. Read tasks/lessons.md --- absorb all past corrections
2. Read tasks/fix_plan.md --- find the current item
3. Read tasks/todo.md --- check session-level plan
4. Confirm scope with user if anything is ambiguous
5. Only then begin work
```

The checklist exists because agents, like humans, have a bias toward action. Without an explicit instruction to orient first, the agent will begin implementing immediately. The checklist converts orientation from a polite suggestion into a contract term—something the agent has been told it *must* do, in writing, before every session.

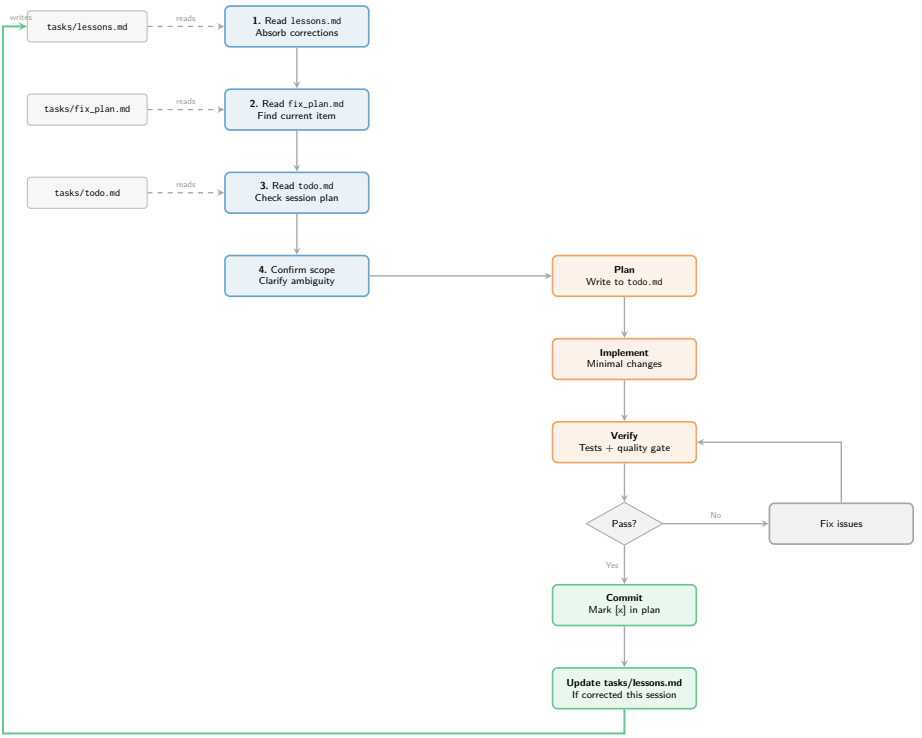


Figure 5.1: The behavior contract loop. Every session begins with an orientation phase—reading `tasks/lessons.md` and `tasks/fix_plan.md` before any work starts. Corrections are written back to `tasks/lessons.md`, creating compound cross-session learning. The file reads at session start are the mechanism through which past corrections influence future behavior.

5.2.5 The Self-Improvement Loop in Detail

The self-improvement loop is the most powerful feature of the behavior contract and the most easily skipped. It requires the agent to maintain a `tasks/lessons.md` file that captures structured patterns from every correction received.

Each lesson follows a structured format that enforces specificity:

```
## Lesson N --- [Short Title]
**What happened**: Brief description of the mistake
**Root cause**: Why it happened
**Rule**: The specific rule to follow from now on
**Date**: YYYY-MM-DD
```

The power of this format is in the specificity requirement. "Be more careful" is not a lesson. "Always run `npm run typecheck` before marking a task complete, because TypeScript errors do not appear in test output for this project" is a lesson. The rule must be concrete enough that a different agent instance, reading it cold, would demonstrably change its behavior.

Over a project's lifetime, `tasks/lessons.md` becomes a project-specific tuning file—the accumulated record of your build system's quirks, the failure modes your agent tends toward, the patterns your codebase uses that differ from common convention.

Key Point

A lessons file that says "be thorough" has zero value. A lessons file that says "this repo uses `pnpm`, not `npm`—running `npm install` corrupts the lockfile and every subsequent verification step will fail" has enormous value. The specificity is the point. Vague lessons give the agent permission to interpret them leniently; specific rules close that gap.

5.3 The VS Code Manual Loop

The VS Code manual loop is the editor-native approximation of the bash Ralph Loop from Chapter 4. It achieves the same structural property—fresh context per work item—through a disciplined manual process: one work item, one chat session, one commit, then a new session.

The word "manual" should not be mistaken for "weak." The manual loop can process complex multi-step projects with the same reliability as the bash version. The cost is friction at session boundaries: instead of a bash script spawning a fresh process automatically, you open a new chat and paste a prompt. The shell helper described in Section 5.5 reduces this to one command and one clipboard paste.

5.3.1 The Fresh-Context Principle

The single most important rule of the VS Code manual loop is this: *never reuse a chat session across work items*.

Reusing a session is tempting. The agent just finished one item; it has context loaded; it feels efficient to say "now do the next one." In practice, this degrades predictably. The agent's context fills with the history of everything it has done, and its attention must span an ever-larger window. By item five or six in a reused session, you are getting a measurably different—and worse—agent than the one that handled item one.

Starting a new session forces the agent to re-orient from files, which is slower per session but consistent across sessions. The quality of item fifteen in a fresh-session loop is the same as item one. The quality of item fifteen in a reused session is not.

Warning

The stop-hook pattern described in Section 4.3.2 of Chapter 4—where a plugin intercepts the agent's exit and redirects it to continue in the same session—exhibits exactly this degradation. It is convenient for short tasks but should not be used for multi-item projects. The fresh-session approach trades session-launch automation for per-session reliability.

5.3.2 The Fix Plan as Canonical State

If conversation history cannot serve as a memory mechanism—because it disappears at every session boundary—all state must live in files. The **fix plan** (`tasks/fix_plan.md`) is the canonical record of what has been completed and what remains. It uses the same checkbox syntax as the bash Ralph Loop:

```
- [ ] Incomplete item
- [x] Completed item
```

The fix plan has one critical structural requirement: the agent reads it at the start of each session to determine what to work on next. This means the file must be self-describing. A new agent instance, reading only this file and the project's contract, must be able to determine: which item to work on, what "done" looks like for that item, and where to look for additional context.

A well-structured fix plan item carries all of this:

```
- [ ] Add user authentication middleware
  - Intercept all requests to /api/* routes
  - Verify JWT token from Authorization header
  - Return 401 with JSON error on invalid or missing token
  - Write integration tests: valid token, expired token,
    missing header, malformed header
```

```
  - Quality gate: npm test && npm run typecheck
```

The item title is a short summary. Sub-items are acceptance criteria. The final line specifies the quality gate. This structure gives the agent everything needed to implement and verify the item without asking clarifying questions—each of which costs a full session round-trip.

5.3.3 The Reusable Prompt Pattern

The reusable prompt is the VS Code equivalent of the bash Ralph `PROMPT.md`: a fixed text the agent receives at the start of every session. Rather than having a bash script feed it automatically, you store it in `tasks/ralph_prompt.md` and paste it at session start (with clipboard assistance from the shell helper).

The prompt structure follows a deliberate order:

1. **Load memory first.** The agent reads `tasks/lessons.md` before reading the task list, ensuring past corrections inform both task selection and implementation approach.
2. **Select one item.** The agent explicitly identifies which item it will work on and states this aloud before planning.
3. **Plan before coding.** The agent writes an implementation plan to `tasks/todo.md` before writing any source code.
4. **Implement and verify.** Standard implementation cycle with quality gate enforcement.
5. **Commit and close.** Mark complete in the fix plan file, commit with a descriptive message, then stop.
6. **Capture lessons.** Any correction received during the session is written to the lessons file before the agent stops.

The explicit stop instruction—"do not continue to the next item"—deserves emphasis. Agents have a natural completion bias that leads them to keep making progress when more items are visible. Without an explicit stop instruction, a single session will process multiple items, accumulating context and violating the one-item constraint that keeps quality stable.

Key Point

The prompt tells the agent to stop not because you do not want more work done, but because you want the next item done at the same quality level as this one. Stopping and restarting with a fresh context is a quality mechanism, not a constraint on throughput.

5.3.4 VS Code Configuration

Two configuration changes make the manual loop practical for real projects. First, the correct mode must be selected: not Ask mode (which only suggests code and cannot run terminal commands) and not Edit mode (which cannot run the test suite autonomously).

Agent Mode is required. It is selected from the mode dropdown at the top of the Copilot Chat panel.

Second, the maximum request limit must be raised. VS Code pauses the agent after a default number of tool calls and waits for the user to click Continue. For a complete implementation-verify-commit cycle, this limit must be high enough that the agent never stops mid-task:

```
{
  "chat.agent.maxRequests": 200,
  "github.copilot.chat.agent.runTasks": true
}
```

The second setting is equally important: without it, VS Code may classify certain terminal commands as "tasks" and refuse to run them autonomously. Both settings together allow the agent to work through an entire session without pausing for human confirmation.

5.4 The Combined Approach

The behavior contract and the VS Code manual loop solve different problems, but they compose naturally. The contract specifies how the agent should behave in any given session; the manual loop ensures each session starts with full context and no accumulated history. Together they produce a system with properties that neither approach achieves alone.

Table 5.1: What Each Component Contributes to the Combined System

Component	Problem Solved	Without It
Behavior contract (AGENT.md)	Behavioral quality: plans before coding, verifies before done, learns from mistakes	Agent cuts corners, skips verification, repeats past errors
New session per item	Context management: prevents quality degradation across a long-running session	Quality drops noticeably by item 5–6 in a reused session
`tasks/fix_plan.md`	Canonical task state across session boundaries	Progress tracked in conversation history; lost when session closes
`tasks/lessons.md`	Cross-session learning: mistakes corrected once, not repeatedly	Each session starts from zero; same errors recur indefinitely
`tasks/ralph_prompt.md`	Consistent loop trigger: every session starts identically	Session start-up quality varies; some sessions skip checklist steps

5.4.1 The File System as External Memory

The central architectural insight of the combined approach is treating the file system as the agent's **external memory**. An AI agent's internal memory—its context window—is

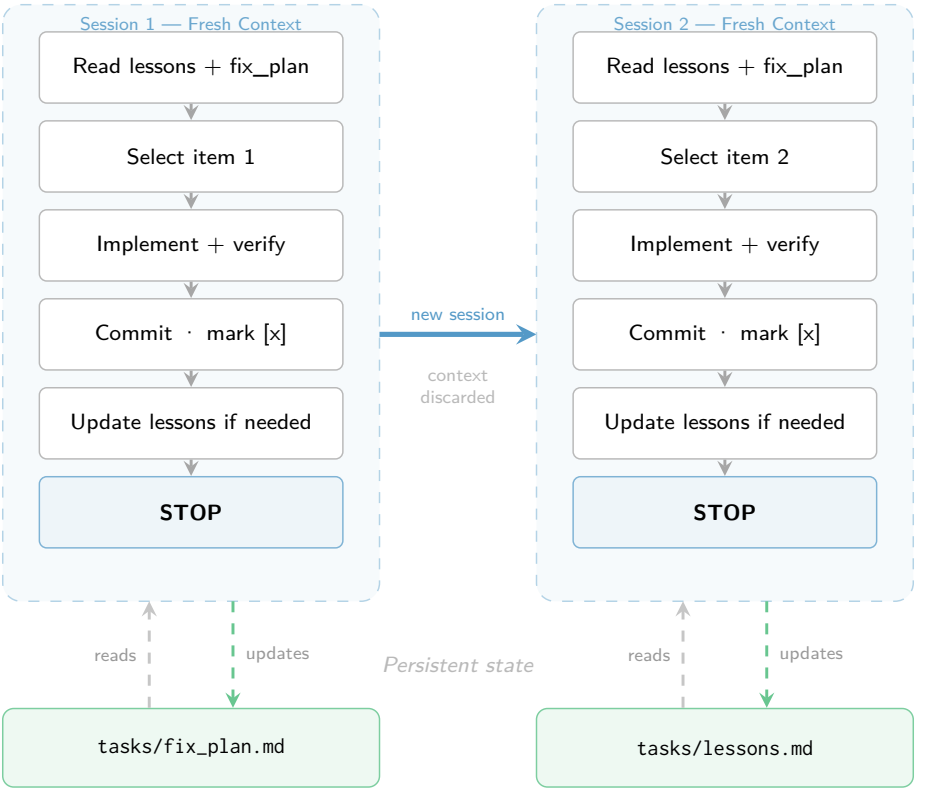


Figure 5.2: The VS Code manual loop. Each session operates in complete isolation with no shared conversation history. All state is persisted to files before the session ends. Each new session reads those files at start-up and re-orients from scratch—the structural equivalent of the fresh-process guarantee in the bash Ralph Loop (see Chapter 4, Section 4.1).

ephemeral. A new session starts with no recollection of what preceded it. Files on disk persist indefinitely.

This means every piece of information the agent needs to function well across sessions must live in a file:

- **What to do next**: `tasks/fix_plan.md`
- **What was learned**: `tasks/lessons.md`
- **How to behave**: `AGENT.md` (or `.github/copilot-instructions.md` for Copilot)
- **The current session plan**: `tasks/todo.md`
- **How to start a session**: `tasks/ralph_prompt.md`

The conversation history becomes disposable. Closing a session at its end is not a loss of information—it is a deliberate design choice. The durable information was written to files before the session ended. The context window was only ever a working scratchpad.

This requires a significant mindset shift for developers accustomed to treating long chat sessions as productive. In the combined approach, a long session is a warning sign. Sessions should be short, focused on exactly one item, and followed immediately by file updates and a commit.

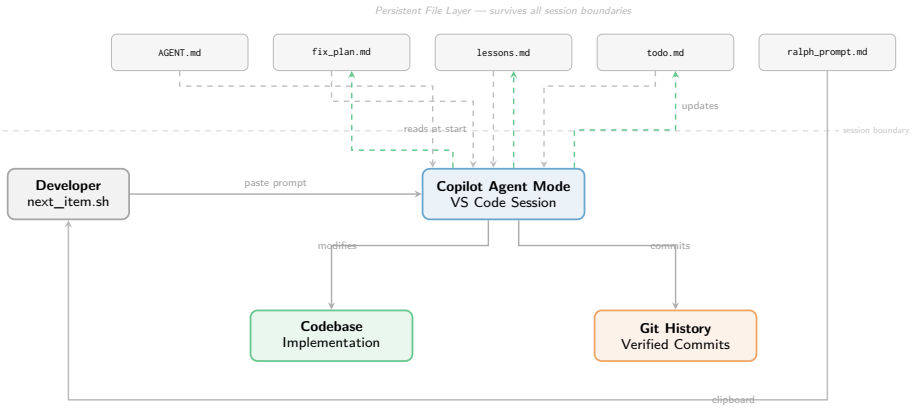


Figure 5.3: Combined system architecture. The persistent file layer survives all session boundaries. The Copilot Agent session reads the file layer at start-up, implements one work item, writes updates back to files, commits, and stops. The developer runs `next_item.sh` to prepare each session. Only the agent session itself is ephemeral; all durable state lives in files.

5.4.2 The Relationship Between Components

The components are not redundant—each covers ground the others do not. The behavior contract is read by the agent automatically, without any prompt instruction; it establishes the baseline for the entire project. `tasks/ralph_prompt.md` is pasted manually and triggers

a specific session protocol—it is the activation key for the loop. `tasks/fix_plan.md` answers "what is the next task?"; `tasks/lessons.md` answers "what do I know that should change how I approach it?" None of these can substitute for any other.

The relationship is captured by a simple rule: *state that must survive session boundaries lives in a file; state that exists only to serve the current session may live in context*. By that rule, task status, learned patterns, session plans, and behavioral contracts are all files. The working scratchpad of the current implementation step can live in context and will be discarded without loss when the session ends.

5.5 Your First Combined Session

This walkthrough builds a working combined system from scratch using VS Code with GitHub Copilot (an active Copilot subscription is required) and a project with an existing test suite.

5.5.1 One-Time Setup

Run these commands once per project to create the required scaffolding:

```
mkdir -p tasks scripts .vscode .github
touch tasks/fix_plan.md tasks/lessons.md
touch tasks/todo.md tasks/ralph_prompt.md
touch .github/copilot-instructions.md
touch scripts/next_item.sh
chmod +x scripts/next_item.sh
git add -A
git commit -m "chore: add agent contract scaffolding"
```

5.5.2 Populate the Behavior Contract

For GitHub Copilot, the behavior contract lives in `.github/copilot-instructions.md`. For Claude Code, copy the same content to `CLAUDE.md`; for Amp, to `AGENTS.md`.

```
# Behavior Contract
# GitHub Copilot reads this file automatically in Agent Mode.

## Session Start --- Do This First, Every Time

Before touching any code:
1. Read tasks/lessons.md completely
2. Read tasks/fix_plan.md --- identify the current item
3. Confirm scope before beginning
4. Only then write code

## Loop Contract

Work on exactly ONE item per session.
```

```
Write plan to tasks/todo.md before coding.
Verify with the item's quality gate before marking done.
Commit on success. STOP. Do not continue to next item.

## Behavioral Rules

### 1. Plan Mode Default
- Enter plan mode for any 3+ step task or architectural decision
- If something goes sideways: stop and re-plan, do not push through
- Write plans as checkable items in tasks/todo.md

### 2. Subagent Strategy
- Use subagents for research, exploration, parallel analysis
- Primary context window acts as a scheduler only
- One task per subagent for focused execution

### 3. Self-Improvement Loop
- After ANY correction: update tasks/lessons.md immediately
- Format: ## Lesson N --- [title] / what happened / root cause /
    rule
- Review tasks/lessons.md at every session start (see above)

### 4. Verification Before Done
- Never mark [x] without running the quality gate
- Ask: would a staff engineer approve this?
- Run tests, check logs, demonstrate correctness

### 5. Demand Elegance (Balanced)
- For non-trivial changes: ask "is there a more elegant way?"
- Skip this for simple, obvious fixes --- do not over-engineer
- If a fix feels hacky, implement the clean solution instead

### 6. Autonomous Bug Fixing
- When given a bug report: fix it without asking for hand-holding
- Consult logs, failing tests --- then resolve
- Zero context switching required from the user

## Core Principles

- Simplicity: the minimal change that correctly solves the problem
- No Laziness: find root causes; no temporary fixes
- Files Over Memory: all durable state lives in files, not chat
```

5.5.3 Populate tasks/fix_plan.md

Replace the template items with your actual work. Acceptance criteria and quality gates are not optional—they are what allow the agent to verify its own work without asking you:

```
# Fix Plan
#
```

```
# Single source of truth for task progress.
# Agent reads this at session start.
# Never track progress in chat history --- only here.

## Agent Instructions
- Read tasks/lessons.md FIRST, before reading this file
- Work on the FIRST incomplete item only
- Write implementation plan to tasks/todo.md before coding
- After completing: mark [x], run quality gate, commit, STOP

## Items

- [ ] Your first task
  - Acceptance criterion 1
  - Acceptance criterion 2
  - Quality gate: npm test && npm run typecheck

- [ ] Your second task
  - Acceptance criterion 1
  - Quality gate: npm test

## Completed
# Completed items move here with a one-line note and date
```

5.5.4 Populate tasks/lessons.md

Bootstrap the lessons file with the one lesson every project starts with:

```
# Agent Lessons
# Read this file completely before starting any work session.
# Add a new lesson after any correction using the format below.

## Format
## Lesson N --- [Short Title]
**What happened**: What the agent did wrong
**Root cause**: Why it happened
**Rule**: The specific rule to follow from now on
**Date**: YYYY-MM-DD

---

## Lesson 0 --- Always Read This File First
**What happened**: Bootstrap lesson --- no specific incident
**Root cause**: Agents lose cross-session context by default
**Rule**: Read this file completely before doing any work in
          a new session. No exceptions.
**Date**: (project start)
```

5.5.5 Populate tasks/ralph_prompt.md

This is the text pasted into every new chat session. The ordering of steps is deliberate and should not be rearranged:

```
You are starting a new work session.
Follow these steps in order, without skipping any.

STEP 1 --- LOAD MEMORY
Read tasks/lessons.md completely.
Acknowledge any lessons relevant to today's work.

STEP 2 --- SELECT TASK
Read tasks/fix_plan.md.
State which item you will work on (first unchecked - [ ]).

STEP 3 --- PLAN
Write your implementation plan to tasks/todo.md as checkable
items before writing any source code.
Review the plan: is there a simpler approach?

STEP 4 --- IMPLEMENT
Execute the plan. Mark steps [x] as you complete them.
Use subagents for research. Keep primary context lean.

STEP 5 --- VERIFY
Run the quality gate specified in the item.
If anything fails: fix, re-run, repeat until green.
Do NOT mark the item done while the quality gate fails.

STEP 6 --- COMMIT AND CLOSE
Mark the item [x] in tasks/fix_plan.md with today's date.
Commit: "fix: [item title]"

STEP 7 --- STOP
Do not work on the next item.
If you received any correction during this session, append
a new lesson to tasks/lessons.md before stopping.
Report what was completed and any observations.
```

5.5.6 Create the Shell Helper

Save `scripts/next_item.sh`. This script reports progress, shows the next item, and copies the session prompt to the clipboard:

```
#!/usr/bin/env bash
set -euo pipefail

PLAN="tasks/fix_plan.md"
PROMPT="tasks/ralph_prompt.md"
LESSONS="tasks/lessons.md"
```

```

echo "========================================"
echo "  VS Code Ralph --- Session Launcher"
echo "========================================"
echo ""

# Verify required files
for f in "$PLAN" "$PROMPT"; do
  [ -f "$f" ] || { echo "Missing: $f --- run setup first."; exit 1; }
done

[ -f "$LESSONS" ] || echo "# Agent Lessons" > "$LESSONS"

# Progress report
DONE=$(grep -c '^\- \[x\]' "$PLAN" 2>/dev/null || true)
REMAINING=$(grep -c '^\- \[ \]' "$PLAN" 2>/dev/null || true)
LESSON_COUNT=$(grep -c '^## Lesson' "$LESSONS" 2>/dev/null || true)

# Normalize: grep -c returns non-zero exit when count is 0 on some
# systems; the || true above preserves 0 correctly via stdout
DONE=${DONE:-0}
REMAINING=${REMAINING:-0}
LESSON_COUNT=${LESSON_COUNT:-0}

echo "Progress : $DONE done, $REMAINING remaining"
echo "Lessons  : $LESSON_COUNT captured"
echo ""

if [ "$REMAINING" -eq 0 ]; then
  echo "All items complete. Review $PLAN for the full summary."
  exit 0
fi

echo "Next item:"
grep '^\- \[ \]' "$PLAN" | head -1 | sed 's/^/  /'
echo ""
echo "Steps:"
echo "  1. Cmd/Ctrl+Shift+P -> GitHub Copilot: Open Chat"
echo "  2. Select AGENT MODE (not Ask, not Edit)"
echo "  3. Paste the prompt (already in clipboard)"
echo "  4. Let the agent run to completion without interrupting"
echo "  5. Close this chat session entirely when done"
echo "  6. Run this script again for the next item"
echo ""
echo "IMPORTANT: Always start a NEW chat session per item."
echo "Reusing a session accumulates context and degrades quality."
echo ""

# Copy prompt to clipboard (cross-platform)
if   command -v pbcopy &>/dev/null; then
```

```
  pbcopy  < "$PROMPT" && echo "Prompt copied (pbcopy --- macOS)"
elif command -v xclip  &>/dev/null; then
  xclip -selection clipboard < "$PROMPT" \
    && echo "Prompt copied (xclip --- Linux)"
elif command -v xsel   &>/dev/null; then
  xsel --clipboard --input  < "$PROMPT" \
    && echo "Prompt copied (xsel --- Linux)"
elif command -v clip   &>/dev/null; then
  clip < "$PROMPT" && echo "Prompt copied (clip --- Windows)"
else
  echo "No clipboard tool found. Copy the prompt manually:"
  echo "---"
  cat "$PROMPT"
fi
```

5.5.7 Configure VS Code

Create or update `.vscode/settings.json`:

```
{
  "chat.agent.maxRequests": 200,
  "github.copilot.chat.agent.runTasks": true
}
```

5.5.8 Run Your First Session

With setup complete, the per-session workflow is four steps:

```
# Step 1: Run the launcher --- it reports progress and copies prompt
./scripts/next_item.sh

# Step 2: Open a new Copilot chat
# Cmd/Ctrl+Shift+P -> "GitHub Copilot: Open Chat"
# Select Agent Mode from the dropdown (not Ask, not Edit)

# Step 3: Paste the prompt and press Enter
# Watch the agent read lessons, select the item, plan,
# implement, verify, and commit --- without further input.

# Step 4: When the agent stops, verify:
cat tasks/fix_plan.md    # First item should be marked [x]
git log --oneline -3     # Commit should appear
cat tasks/lessons.md     # Any new lessons from this session?

# Then close the chat session entirely and repeat from Step 1.
```

The first session will almost certainly surface configuration issues: the agent may use the wrong test command, reach for a package manager you do not use, or misread an acceptance criterion. This is expected. After each failure, the agent should add a lesson

capturing the correct behavior. By session three, these early mistakes will have disappeared from its behavioral profile.

Warning

Resist the temptation to correct the agent mid-session by typing additional instructions. Doing so accumulates context and trains you into a hand-holding pattern that the loop is designed to eliminate. Instead, let the session fail, review `tasks/todo.md` to understand what went wrong, improve the item spec in `tasks/fix_plan.md`, run `git reset –hard HEAD` to discard uncommitted changes, and start a fresh session. The discipline of full-session isolation is what keeps quality stable at scale.

5.6 Comparing the Three Approaches

Table 5.2: Comparing the Bash Ralph Loop, VS Code Manual Loop, and Combined Approach

Property	Bash Ralph Loop	VS Code Manual	Combined
Fresh context per item	Yes (new process)	Yes (new session)	Yes (new session)
Behavioral quality rules	Partial (in prompt / config)	None by default	Explicit (contract file)
Cross-session learning	AGENTS.md patterns	None	tasks/lessons.md structured
Automation level	High (fully unattended)	Low (manual paste)	Medium (script + paste)
Self-improvement loop	Implicit	None	Explicit
Requires CLI install	Yes	No	No
Works in VS Code	No	Yes	Yes
Session trigger	Automatic (bash)	Manual (paste)	Semi-manual (script copies prompt)
Completion detection	`<promise>COMPLETE</promise>`	Developer reads fix_plan.md	Script reads fix_plan.md
Best for	Server automation, CI, overnight batches	Quick focused tasks, GUI preference	VS Code users wanting reliability parity with bash Ralph

The combined approach does not surpass the bash Ralph Loop in automation—the bash version remains superior for fully unattended execution and CI integration. What the combined approach offers is *reliability parity* with substantially lower setup friction. For teams already in VS Code with Copilot subscriptions, it requires no additional tooling, no CLI installation, and no context-switching away from the editor where the code lives.

The cost is the manual session trigger: approximately thirty seconds per item when the shell helper is in place. For a twenty-item project, that is ten minutes of trigger overhead against hours of autonomous implementation. Most practitioners find this acceptable.

Key Point

Choose the bash Ralph Loop for full autonomy: overnight batch jobs, CI integration, large migrations where zero human interaction is required. Choose the combined approach for active development sessions where you are present at the keyboard and working in VS Code. The underlying principles are identical—fresh context, one item per iteration, state in files. Only the session-trigger mechanism differs.

5.7 Tuning the System

Like the bash Ralph Loop, the combined system requires tuning. The behavior contract and prompt you start with will not be the ones you end with. Huntley's description of the prompt as something that "evolves through continual tuning based on observation of LLM behaviour" applies equally here—the contract is a living document, not a one-time configuration.

5.7.1 Quality Gate Specification

The most common failure in early sessions is the agent running the wrong quality gate. If your project uses `yarn`, `pnpm`, or a custom test script, neither the prompt nor the behavior contract will know this unless you write it explicitly. Add a dedicated section to your contract:

```
## Quality Gates
# Exact commands for this project. Use these and no others.
- Type check : npx tsc --noEmit
- Lint       : npx eslint src --ext .ts,.tsx --max-warnings 0
- Test       : pnpm test --run
- All three must pass before committing any change.
```

After any session where the agent used a wrong command and received a correction, that correction should produce a lesson in `tasks/lessons.md` specifying the correct commands. The lesson ensures future sessions do not repeat the same mistake even if the agent does not re-read the contract carefully.

5.7.2 Item Granularity

Items that are too large will cause the agent to fill the context window before reaching the verification step, producing partial implementations. Items that are too small create session-boundary overhead that slows progress unnecessarily. The right granularity is roughly equivalent to a focused pull request: one coherent unit of work that can be implemented, verified, and committed in a single session. When in doubt, prefer smaller items—they keep context lean and quality consistent, which is the same reasoning behind the one-item-per-loop principle from Chapter 4.

If an item consistently causes the agent to exit without committing, split it. If sessions complete in under five minutes, consider whether adjacent items can be responsibly merged.

5.7.3 tasks/lessons.md Maintenance

Over a long project, `tasks/lessons.md` can accumulate contradictory or stale lessons. A lesson recorded in week one about a dependency that was later removed can actively mislead the agent. Every two to three weeks, spend five minutes reviewing the file: ask the agent to consolidate duplicates, remove stale entries, and reorganize by category. This is low-cost maintenance that prevents the lessons file from becoming a source of confusion rather than clarity.

5.8 Troubleshooting

Table 5.3: Common Issues in the Combined Approach and Their Solutions

Problem	Likely Cause	Solution
Agent ignores contract	Not in Agent Mode	Switch from Ask to Agent Mode in the chat header
Agent continues to next item	No explicit stop instruction	Ensure prompt ends with "STOP. Do not work on next item"
Quality degrades over multiple items	Sessions being reused	Close the chat fully after each item; always start a new session
Agent uses wrong test command	Quality gate not specified	Add exact commands to contract under a Quality Gates section
Contract file not being read	Wrong file location for this tool	Confirm which file your tool reads; verify with a test instruction
tasks/lessons.md not being read	Checklist step skipped	Add a lesson: "Always read tasks/lessons.md before any work"
Item marked done without testing	Verification step skipped	Add to contract: "Never mark [x] without running quality gate"
Session pauses mid-task	maxRequests too low	Set `chat.agent.maxRequests` to 200 in settings.json
Agent asks clarifying questions	Item spec too vague	Add acceptance criteria and quality gate to every item
Same mistake on consecutive days	Lesson not captured or too vague	Ask agent to add or sharpen the lesson; demand specificity
tasks/fix_plan.md not updated	Agent stopped before commit	Run `git reset -hard HEAD`; improve item spec; retry

5.8.1 When the Agent Goes Off Track

The VS Code equivalent of Huntley's "reset and regenerate" guidance from Chapter 4 applies here too. If a session produces a clearly wrong result—incorrect architecture, broken tests, misunderstood requirements—the correct response follows a fixed sequence:

1. **Reset**: Run `git reset -hard HEAD` to discard uncommitted changes.
2. **Diagnose**: Read `tasks/todo.md` to understand what the agent planned versus what it should have planned.
3. **Improve**: Rewrite the item spec in `tasks/fix_plan.md` with clearer acceptance criteria or a more precise quality gate.
4. **Capture**: Add a lesson to `tasks/lessons.md` documenting why the misunderstanding occurred and what the correct interpretation is.
5. **Retry**: Run `next_item.sh`, open a new chat, and start a fresh session.

Do not attempt to salvage a bad session by continuing in the same chat with corrective instructions. The accumulated context of the failed attempt will interfere with the fresh attempt. You are no longer talking to a clean agent; you are talking to one that has already committed to a wrong interpretation and is now reasoning about how to reconcile corrections with it. Reset, capture the lesson, restart.

Chapter Summary

- Autonomous AI development in VS Code has two independent problems: **behavioral quality** (what the agent does within a session) and **context management** (how quality holds across session boundaries). Each requires its own solution.
- A **behavior contract** (AGENT.md for most agents, `.github/copilot-instructions.md` for Copilot) is read automatically at every session start. It encodes six behavioral rules, a mandatory orientation checklist, and a task management protocol.
- The **self-improvement loop** is the contract's most consequential feature. Every correction writes a structured lesson to `tasks/lessons.md`; every session reads that file first. Mistakes are corrected once and do not recur.
- The **VS Code manual loop** achieves the same structural property as the bash Ralph Loop—fresh context per work item—through disciplined session management: one item, one session, one commit, then a new session.
- **Never reuse a chat session across work items.** Quality degrades predictably with context accumulation. Fresh-session discipline is a quality mechanism, not a throughput constraint.
- The **file system is the agent's external memory**. Task state, learned patterns, behavioral rules, and session plans all live in files. Conversation history is ephemeral and should be treated as disposable.

- The **combined approach** delivers reliability parity with the bash Ralph Loop for VS Code users. The cost is a manual thirty-second session trigger per item; the benefit is no additional tooling beyond VS Code and a Copilot subscription.
- **Quality gates must be specified explicitly** in the contract and in each fix_plan item. An agent that does not know your exact test command cannot self-verify—and an agent that cannot self-verify will declare tasks done prematurely.
- **Confirm which file your tool reads automatically** before deploying the contract. A contract in the wrong location is silently ignored, with no error and no warning.
- **Tune continuously.** The contract, the prompt, and `tasks/lessons.md` are living documents. The versions you start with will not be the versions you end with. Treat tuning as part of the work.

What's Next: You now have two implementations of the Ralph Loop pattern—the bash loop from Chapter 4 for unattended execution, and the combined approach from this chapter for editor-native work. Both depend on something neither provides: high-quality task decomposition. Chapter 6 addresses how to break features into atomic, verifiable units that AI can implement reliably—specifications that leave no room for misinterpretation, and acceptance criteria that a machine can actually verify.

Parts II–III — Chapters Not Included

Chapter 6: Task Decomposition for Machines *Not included in this sample.*

Chapter 7: Memory and Context Engineering *Not included in this sample.*

Chapter 8: Quality Gates and Backpressure *Not included in this sample.*

Chapter 9: Advanced Prompting for Autonomous Systems *Not included in this sample.*

Chapter 10: Agentic Patterns in Practice *Not included in this sample.*

Chapter 11: Scheduling and Overnight Execution *Not included in this sample.*

Chapter 12: Multi-Agent Orchestration *Not included in this sample.*

Chapter 13: Safety, Security, and Risk Management *Not included in this sample.*

Appendices — Not Included

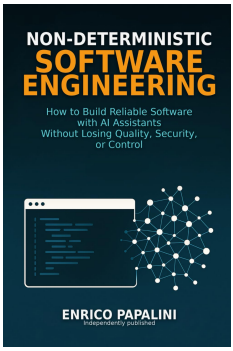
Appendix A: Complete Implementation Guide *Not included in this sample.*

Appendix B: Tool Reference *Not included in this sample.*

Appendix C: Troubleshooting and Debugging *Not included in this sample.*

Appendix D: Ralph Loops with VS Code and GitHub Copilot *Not included in this sample.*

The Complete Series



Non-Deterministic Software Engineering

How to Build Reliable Software with AI Assistants

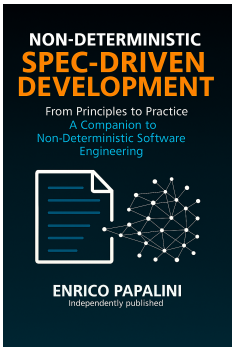
The foundational guide to AI-assisted software development. Covers enterprise workflows, security frameworks, team dynamics, quality measurement, and the eighteen patterns that separate successful AI adoption from expensive chaos.

Inside you'll discover:

- The Review Gauntlet—multi-layer verification for AI-generated code
- Security frameworks designed for non-deterministic systems
- Team structures that amplify AI capabilities
- Measurement systems that actually predict outcomes

https://www.amazon.com/dp/B0G7LPJBTH

Also in the Series



Non-Deterministic Spec-Driven Development

How to Write Specifications That AI Can Actually Execute

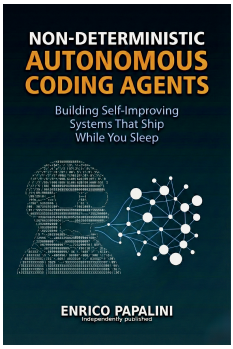
The specification companion that bridges intent and implementation. Learn the five principles for effective AI specifications, the four-phase process with explicit gates, and integration with modern tooling like Spec Kit.

Inside you'll discover:

- Why specifications matter more than ever in the AI era
- The constitution, clarification, and consistency framework
- Six playbooks for different development scenarios
- Six new patterns extending the main book's catalog

https://www.amazon.com/dp/B0GGZ44RNX

Get the Complete Book



Non-Deterministic Autonomous Coding Agents

Building Self-Improving Systems That Ship While You Sleep

What if you could wake up to production-ready code, tested and committed, while you slept? This is the definitive guide to building AI systems that work independently—picking up tasks, implementing solutions, running tests, and committing code without human intervention.

What you'll master:

- The Ralph Loop Pattern—continuous cycles of read-plan-implement-validate-commit
- Task decomposition, memory engineering, and quality gates
- Advanced prompting: reverse prompting, constraint cascades, role stacking, and verification loops
- Scheduling and overnight execution on macOS, Linux, and Windows
- Multi-agent orchestration: hat-based, planner-worker, and parallel agents
- Safety, sandboxing, and blast radius containment

231 working examples · **97 agentic patterns** · Claude Code, GitHub Copilot & Amp

https://www.amazon.com/dp/B0GMCFLH2C